

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 6 — Projects e Context Engineering

Che cosa scriverci, e che cosa no

Scrivete ciò che Claude non vede

La regola di Anthropic per un `CLAUDE.md`, e due domande prima di ogni riga

- Un `CLAUDE.md` si carica **a inizio sessione**, per intero, e pesa su **ogni richiesta**: lo dice la tabella della documentazione. Che cosa scriverci, allora?
- Dalle best practice di Claude Code: «Only write down what Claude **can't find** by reading the repo, or what it would otherwise **spend time rediscovering** every session.»
- Ciò che non trova è ciò che nel repository **non è mai entrato**: «A decision your team made in a meeting», «A deploy process that lives in one person's head», «The environment variable your integration tests need»
- Prima di scrivere una riga, **due domande**: «Could it find this by reading the code?» e «Would it actually go and look?». E poi: «If both are yes, leave it out.»

Scrivete solo ciò che Claude non troverebbe, o non andrebbe a cercare.

Che cosa tenere

Le righe che passano le due domande, e due esempi del corso

- **Come verificare il lavoro**: «How to confirm work in this repo: the exact test, typecheck, and lint commands, and how to run a single test file»
- **Dove il codice inganna**: «an old pattern that's everywhere but that new code must not copy, or a directory that looks unused but isn't»
- **Le decisioni fuori dal repository**: «Decisions that aren't in the repo: which error helper the team settled on, how deploys work, what needs a migration review»
- **Ciò che avete corretto due volte**: fra i segnali per aggiungere una riga, «Claude makes the same mistake a second time»; e «A repeated mistake or a recurring review comment is a CLAUDE.md edit, not a one-off correction in chat.»
- **La forma**: «Structure: group related instructions under markdown headers and bullets. Organized sections are easier for Claude to follow than dense paragraphs.»

Dalle best practice di Claude Code, dalla pagina sui `CLAUDE.md` e da «Extend Claude Code»

- Nel **laboratorio** del modulo, due tracce sui progetti sintetici del corso: M6-E02 sul **gestionale** PHP di CiaspoLink, M6-E03 sulla sua **rete**. Dal `CLAUDE.md` di riferimento della rete:

```
la rete, la voce dei Comandi, a capo per la slide

- «A posto» vuol dire i due controlli, dalla radice:
  `python3 progetti/rete/genera.py --check` e
  `python3 progetti/rete/genera.py --canone`; si riporta
  l'esito di ciascuno.
```

Che cosa tenere

L’esempio del gestionale: il suo `CLAUDE.md` di riferimento

righe scelte, a capo per la slide; «[…]» segna un taglio

```
[…]
## Comandi

- Dalla cartella `progetti/gestionale-php/`: `../../tools/php.sh
  composer install` se manca `vendor/`, poi `../../tools/php.sh
  phpunit`.

## Convenzioni che il codice non dice

- Ogni commit ha nel messaggio una riga `Rif: GES-<n>`, il numero
  della richiesta nel tracker del team; `Rif: GES-0` per la
  manutenzione, come un refuso.
[…]
## Insidie

- `database/dati/` e `composer.lock` si rigenerano, non si
  modificano: i dati con `python3 genera.py`, le dipendenze con
  Composer.
```

M6-D03 Il comando dei test: Claude andrebbe a cercarlo?

Demo del docente in Claude Code: la stessa richiesta, senza e con quattro righe

- A voce: il comando dei test del gestionale, Claude lo troverebbe? Andrebbe a cercarlo?
- Il comando è già nel **README** del gestionale, sotto «Primo avvio». Due copie usa e getta, una sessione nuova di `claude -p` in ciascuna, da `progetti/gestionale-php`: Sonnet, effort low, Bash consentito. Nel braccio con, in più, un `CLAUDE.md` di quattro righe:

`con-la-riga/CLAUDE.md, nella cartella del gestionale`

```
# Gestionale CiaspoLink

- Test, da questa cartella: `../../tools/php.sh phpunit`.
- Senza `vendor/`, prima: `../../tools/php.sh composer install`.
```

`richiesta.txt, uguale nei due bracci, a capo per la slide`

```
Esegui la suite di test del gestionale e dimmi quanti test passano
e quanti falliscono.
```

- Guardate quali **comandi** esegue Claude, e se apre il README. Il comando della demo è al passo 1 del README in `m6_projects_context/s5_projects_context/m6_4_che_cosa_scriverci/M6-D03_la_riga_che_mancava/`

M6-D03 Il comando dei test: Claude andrebbe a cercarlo?

Demo del docente in Claude Code: la stessa richiesta, senza e con quattro righe

La registrazione del corso: tre esecuzioni per braccio. Il controllo legge dallo stream ogni comando e il suo risultato; per braccio, in tutto:

braccio	la suite ha girato	il README letto	turni	secondi	token
senza	2 su 3	2 su 3	14	30,6	108.791
con	3 su 3	0 su 3	9	23,1	64.822

La risposta: lo troverebbe, il comando è nel README; ci è andato in **2 esecuzioni su 3**, al secondo passo, con un `grep` di «test» nel README, poi con `tools/php.sh`: «OK (44 tests, 107 assertions)». Nella terza no: `which phpunit php composer`, e «Non posso eseguire la suite di test»

Dalle best practice: «Only write down what Claude **can't find** by reading the repo»: nell'esecuzione 3 la suite ha girato con la riga e senza no, **«confermata 1 su 3»**; nella 1 e nella 2 ha girato anche senza, «non distinguibile»

«or what it would otherwise **spend time rediscovering** every session»: i turni, i secondi e i token della tabella sono **dati**, e la frase non dice quanto. Le quattro righe pesano da +223 a +225 token sulla prima richiesta

Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

Corto, concreto, coerente

Come si scrivono le righe che restano

- **Corto.** Nelle best practice: «every line in it takes a little of Claude's attention on every task, whether it's relevant or not». Nella pagina sui `CLAUDE.md`: «target under 200 lines per CLAUDE.md file». E l'Academy: «The leaner the file, the more of it Claude actually follows.»
- **Concreto.** «"Run npm test before committing" instead of "Test your changes"»; «Put new API routes in src/api/handlers, one per file», non «Follow best practices for API routes»
- **Coerente.** «if two instructions contradict each other, Claude may pick one arbitrarily»
- L'**enfasi** non salva una riga: «If Claude keeps ignoring a line that is in CLAUDE.md, adding IMPORTANT probably won't fix it.» I modelli attuali «follow a plainly worded line». Ciò che deve valere sempre lo esegue Claude Code: «a hook, a deny rule, or a managed setting»
- Dal blog di Claude: «Keep your CLAUDE.md lightweight», la maggior parte dei token sulle **insidie** del codice; e niente ovvio: «Avoid stating 'the obvious' things Claude should know by looking at your file system or your repo.»

Che cosa togliere

Ciò che Claude ricava da sé, l'ovvio, i segreti

- In una riga: fuori ciò che Claude **ricava dal codice**, l'**ovvio** e i **segreti**; `/doctor` propone i tagli
- Fra i consigli **superati**, nelle best practice: «Describe your architecture and file layout in CLAUDE.md». Perché: «Claude reads the repo itself. Only write down what isn't in there.»
- `/doctor` propone tagli a un `CLAUDE.md` versionato, dice la documentazione: «it cuts content Claude can derive from the codebase, such as directory layouts, dependency lists, and architecture overviews», e «keeps pitfalls, rationale, and conventions that differ from tool defaults»
- Non vale la pena, per la guida del centro assistenza: «Full API documentation (Claude can read the code directly).», «Changelogs or history.», «Anything that is already obvious from the file tree.»
- Mai i **segreti**, dall'articolo sui `CLAUDE.md`: «Don't include sensitive information, API keys, credentials, database connection strings, or detailed security vulnerability information»
- E Anthropic stessa, sul blog di Claude: «we found that we were overconstraining Claude Code, both through our system prompt and in our CLAUDE.md files and skills»